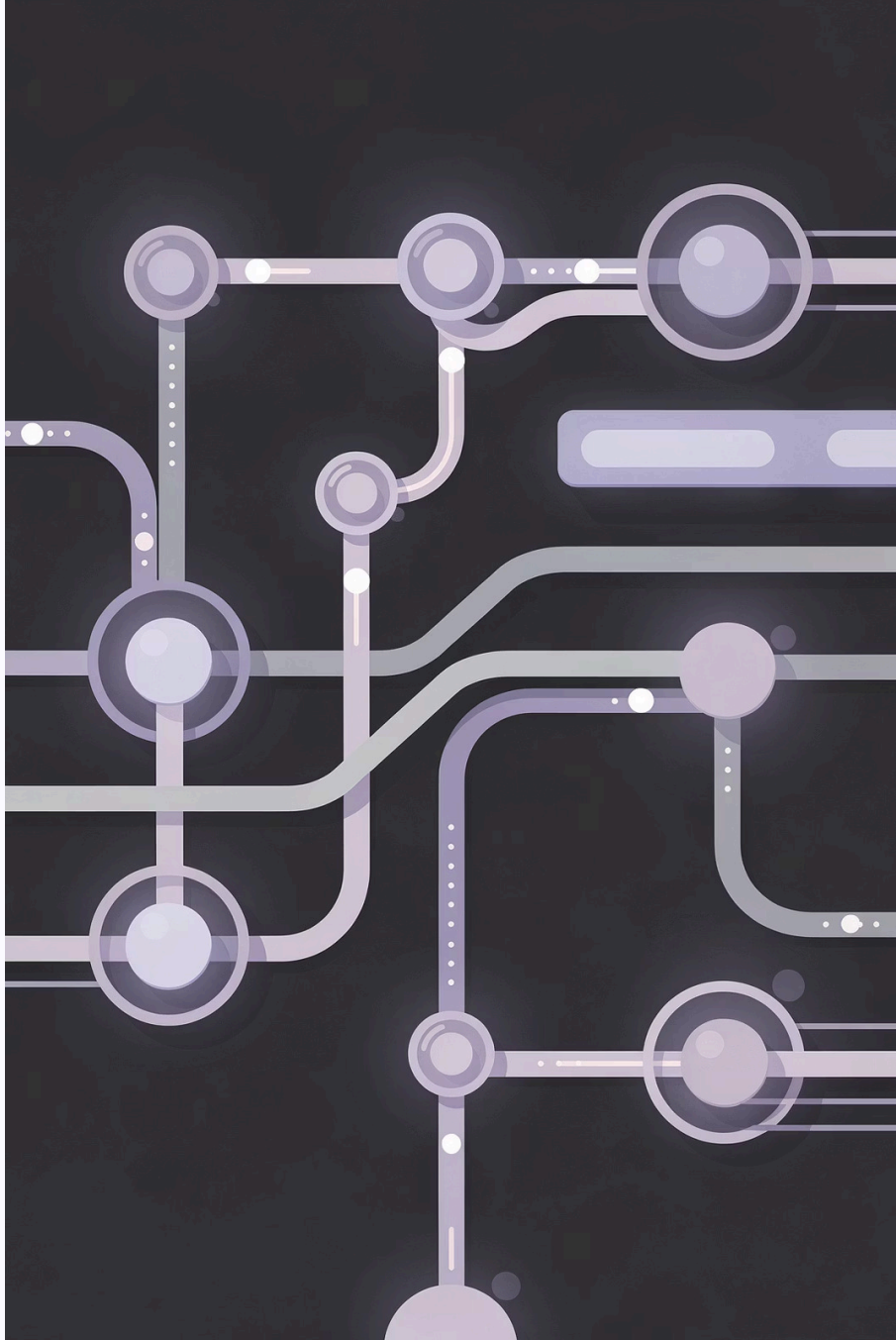
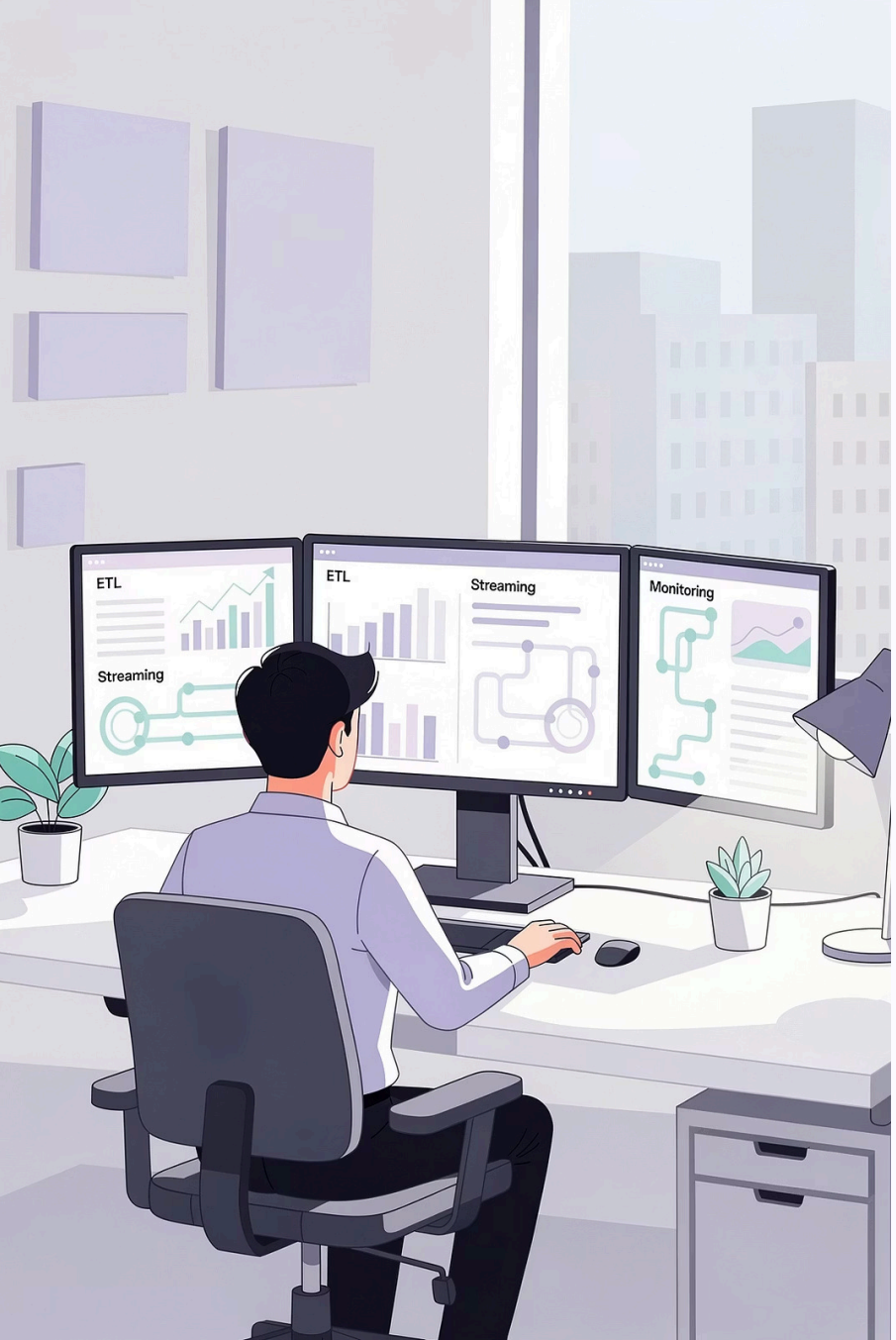
Databricks Workflows

Automating, scheduling, and orchestrating your data pipelines at scale — from raw ingestion to production-ready outputs.

DATA ENGINEERING SERIES





Where We Left Off

Over the past sessions, we've built a solid foundation for working with data on Databricks. Here's a quick recap of what we've covered so far — and where we're headed next.

1

Day 2 — Transformations

Cleaning, reshaping, and enriching data using PySpark and DataFrame operations.

2

Day 4 — Delta Lake

Reliable, versioned storage with ACID transactions and time travel capabilities.

3

Today — Workflows

Putting it all together: **How do we automate pipelines** so they run reliably without manual intervention?

What Is Orchestration?

Orchestration is the practice of coordinating multiple tasks, dependencies, and schedules into a reliable, automated pipeline. Without it, data workflows remain fragile, manual processes. Databricks Workflows provides orchestration natively — no external tools required.

Scheduling

Define when your pipelines run — on a fixed cadence, at specific times, or in response to external triggers. Eliminate manual intervention entirely.

Dependency Management

Ensure tasks execute in the correct order. If Task B depends on Task A, the orchestrator enforces that relationship automatically.

Monitoring

Track run status, inspect logs, receive failure alerts, and audit historical runs — all from a single unified interface.

Workflow Components

A Databricks Workflow is built from three core building blocks. Understanding how they fit together is the foundation for designing robust pipelines.

Jobs

The top-level container for your workflow. A Job defines **what runs, when it runs, and on what infrastructure**. You can trigger jobs manually, on a schedule, or via API.

Tasks

The individual units of work inside a Job. Each Task represents a discrete step in your pipeline — a notebook, a script, a SQL query, or even another Job. Tasks can be chained or run in parallel.

Clusters

The compute layer that executes your tasks. Jobs can spin up **job clusters** (ephemeral, cost-efficient) or reuse existing interactive clusters. Cluster configuration lives at the task level.

Task Types

Databricks Workflows supports a variety of task types, giving you flexibility to compose pipelines from different code artefacts and languages. Choose the type that best fits your team's workflow.

📓 Notebook

The most common task type. Run an interactive Databricks notebook directly within the job. Ideal for exploratory logic that's been promoted to production, or when you need rich inline documentation alongside code.

🐍 Python Script

Execute a `.py` file stored in a repository or DBFS. Best for modular, version-controlled code that follows software engineering practices. Supports full Python packaging and imports.

🗄️ SQL

Run a SQL query or a file against a Databricks SQL warehouse. Perfect for analytics-layer transformations, reporting refreshes, or teams who prefer SQL-first pipelines without writing Python.

Job Cluster vs. Interactive Cluster

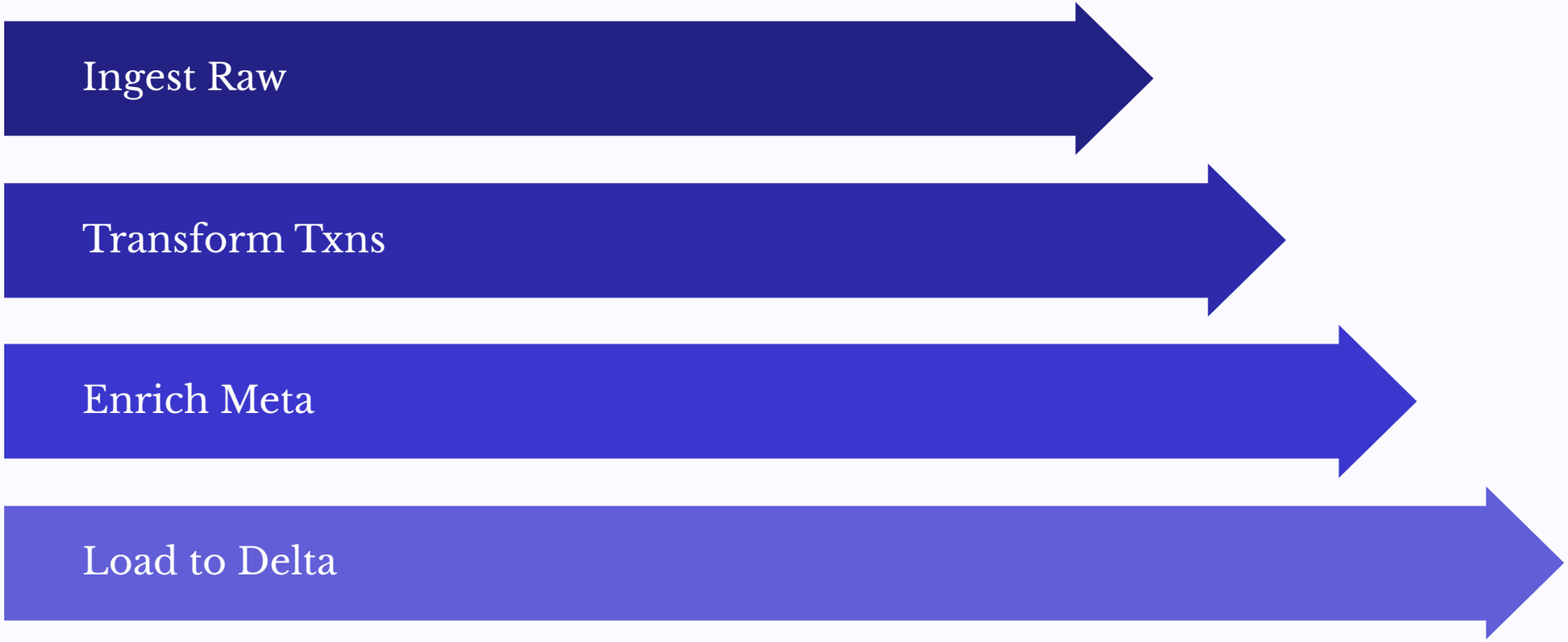
Choosing the right cluster type has a direct impact on cost, reproducibility, and performance. Here's how they compare across the most important dimensions.

Dimension	Job Cluster	Interactive Cluster
Lifecycle	Spun up for the job, terminated on completion	Long-running, shared across sessions
Cost	Pay only for job duration — more efficient	Accrues cost while idle
Reproducibility	Clean environment every run — highly consistent	Shared state can introduce drift
Best For	Scheduled, production pipelines	Development, exploration, ad-hoc work
Startup Time	Slightly slower due to provisioning	Already running — immediate execution

✅ **Best practice:** Always use Job Clusters in production. Reserve Interactive Clusters for development and experimentation.

Workflow DAG — Task Dependencies

At the heart of Databricks Workflows is a **Directed Acyclic Graph (DAG)** — a visual map of tasks and the dependencies between them. The DAG determines execution order and enables parallel processing where possible.



Sequential Execution

Tasks run one after another when downstream steps depend on upstream outputs. Guarantees data is ready before the next step begins.

Parallel Execution

Independent tasks run simultaneously, reducing overall pipeline duration. Databricks automatically schedules parallel branches when no dependency exists between them.

Scheduling Your Workflows

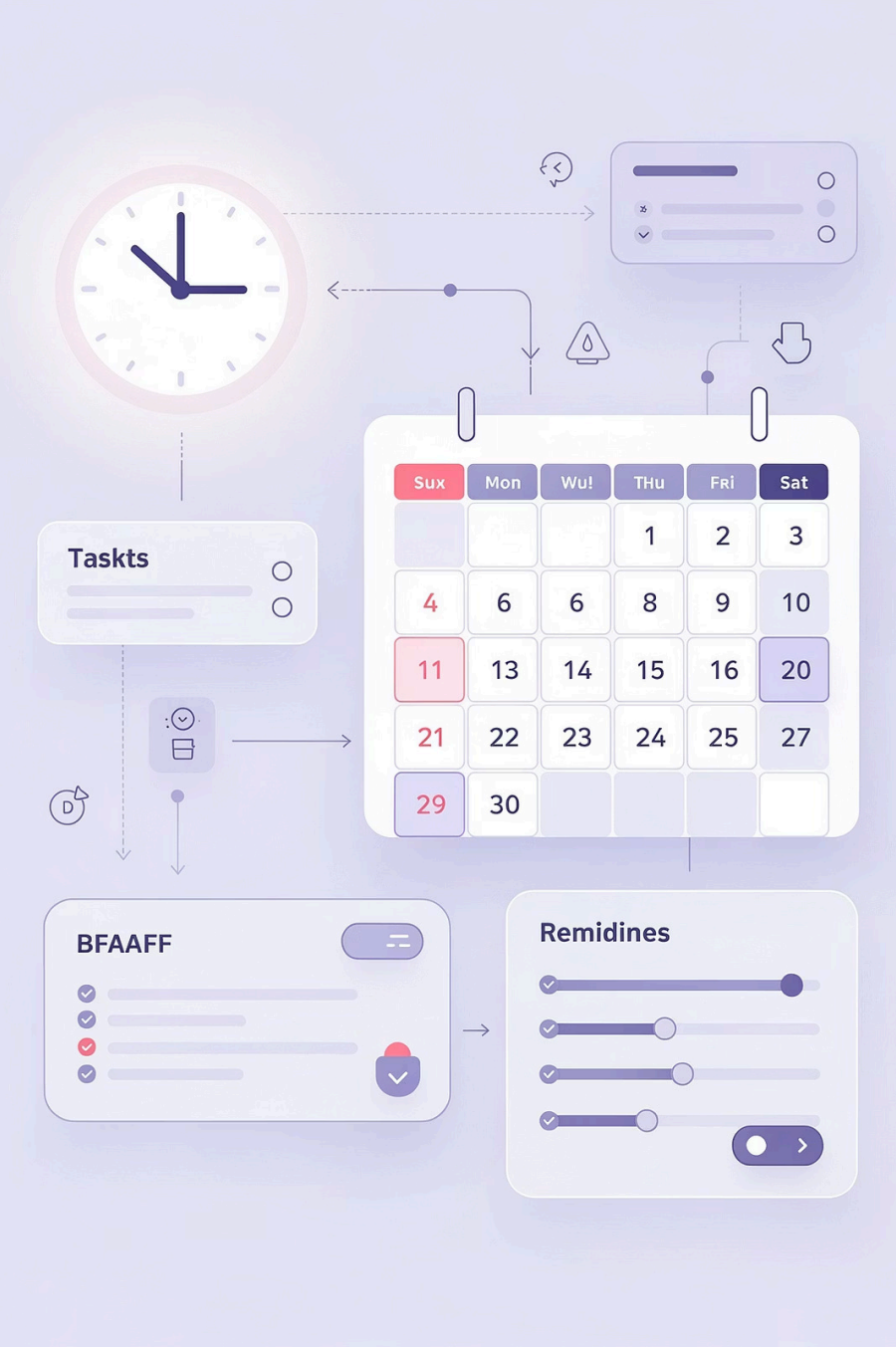
Databricks Workflows supports two primary scheduling paradigms — choose the one that fits your pipeline's data freshness requirements and operational model.

⏰ Cron-Based Scheduling

Define a fixed schedule using standard **cron syntax** (e.g., `0 6 * * *` for 6 AM daily). The UI also provides a plain-English builder to avoid cron mistakes. Best for batch pipelines with predictable cadences — hourly, daily, weekly refreshes.

⚡ Event-Based Triggering

Trigger jobs programmatically via the **Databricks REST API** or in response to external events — file arrivals, upstream job completions, or CI/CD pipeline hooks. Ideal for near-real-time pipelines where waiting for a fixed schedule introduces unacceptable latency.



Monitoring, Logs & Failure Handling

Production pipelines **will** fail — the question is how gracefully you recover. Databricks Workflows provides built-in observability and retry mechanisms so failures are caught, logged, and resolved with minimal manual intervention.

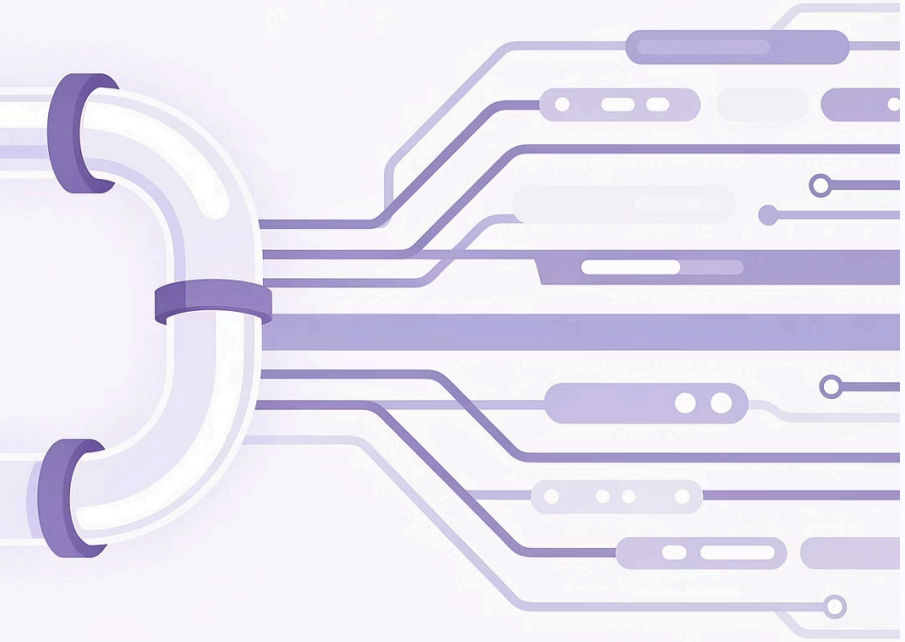
Logs & Observability

Every job run produces structured logs accessible from the Workflow UI. Inspect per-task stdout, stderr, and Spark driver logs. Use the run history timeline to compare successful vs. failed runs and identify patterns.

Failure Handling & Retries

Configure per-task **retry policies** with maximum attempts and delay between retries. Set up **email or webhook alerts** on failure, timeout, or success. Use **task-level dependencies** to define whether downstream tasks should skip, continue, or fail when an upstream task errors.

💡 **Tip:** Combine retries with idempotent task logic — ensure re-running a task produces the same result as running it once. Delta Lake's ACID semantics make this straightforward.



What's Next: Delta Live Tables

You now have a solid understanding of how Databricks Workflows orchestrates pipelines — jobs, tasks, clusters, DAGs, schedules, and monitoring all work together to automate your data engineering.

> 👉 **"Now let's simplify pipelines even further using Delta Live Tables (DLT)."**

→ **Declarative pipeline definitions** — describe *what* your data should look like, not *how* to compute it

→ **Built-in data quality** — enforce expectations and quarantine bad records automatically

→ **Automated orchestration** — DLT manages dependencies, retries, and cluster lifecycle for you